Mudabbir: AI-Assisted Subject-Specific Study of the Quran

FYP blueprint and design notes · draft for team discussion

1. Vision and scope

Mudabbir helps a learner study the Quran by subject (for example patience, justice, family, knowledge). For any topic it finds the relevant verses, ranks them by how much they contribute to that topic, explains them using classical tafsir, and turns the result into study material.

#	Module	What the user gets
1	Semantic and relevance search	Find verses by meaning, not only by exact words
2	Knowledge-weight ranking	Verses ordered by how much knowledge they carry on the topic
3	Summary generation	A grounded topic summary with verse references
4	Khutba generation	A structured Friday-sermon draft for a topic
5	Assessments	Quizzes and tests generated from the topic
6	Micro-credential courses	Short structured courses with a completion badge

Assumption to confirm with your supervisor: “knowledge weight” means the explanatory and thematic value a verse has for a given topic. It is a study-aid score, not a religious ruling about the verse. Section 6 turns it into a measurable score.

2. Golden rules (put these in your report)

1. The Quran text always comes from a verified database. The language model never types a verse from memory; code inserts the verse using its verse key (for example 2:153).
2. Every claim in generated content cites a verse key and, where used, the tafsir source.
3. If retrieval is weak, the system says there is not enough evidence instead of guessing.
4. Hadith is used only from graded collections with source and grading shown. If you cannot do this reliably, leave hadith out of version 1.

5. Mudabbir is a study aid, not a fatwa engine. It gives no rulings and shows a clear disclaimer.
6. A qualified person reviews the gold data and samples of generated output (summaries, khutbas, quizzes).

3. System architecture

```
Data layer    Arabic text · translations · tafsir · morphology and roots
     ↓
Index layer   BM25 index + vector index + metadata (surah, juz, topic tags)
     ↓
Retrieval     Hybrid search → rerank → knowledge-weight score → tiers
     ↓
Generation    Summary · Khutba · Assessment · Course builder
              (LLM sees only the retrieved verses and tafsir)
     ↓
Guardrails    Verse-key insertion · citation check · refuse on weak evidence
     ↓
App           Web UI · REST API · optional Mudabbir MCP server
```

4. Data sources

Source	Use	Link
Tanzil	Arabic text and many translations (check the license: text must stay unaltered, with attribution)	https://tanzil.net
Quranic Arabic Corpus	Word-by-word morphology, roots, concept ontology	https://corpus.quran.com
Quran.com (Quran Foundation)	Text, translations, tafsir	https://quran.com
quran-mcp (official)	Verified text, translations and tafsir through MCP; useful for cross-checking	https://github.com/quran/quran-mcp
tafsir-mcp	Five classical tafsirs and word-level linguistic data	https://github.com/tafsircenter/tafsir-mcp

Source	Use	Link
Hugging Face datasets	Existing Quran datasets (check quality and license before use)	https://huggingface.co/datasets?search=quran

Suggested tables:

- verse: verse_key, surah, ayah, arabic_uthmani, arabic_clean, translation_en, translation_ur, roots, juz
- tafsir_passage: tafsir_id, verse_key, text, source, language
- topic: topic_id, name, synonyms, parent_topic
- verse_topic: verse_key, topic_id, label_source (scholar, model), grade (0 to 3)

5. Module 1: Semantic and relevance search

Semantic search finds verses that mean something similar to the query. Relevance ranking then decides which of those best answer the topic. Build both, and keep keyword search as your baseline.

Pipeline:

1. Query understanding: detect language (English, Urdu, Arabic), normalize Arabic, expand with synonyms and roots.
2. Lexical retrieval: BM25 over Arabic clean text, translations and tafsir.
3. Dense retrieval: embeddings of verse plus translation plus a tafsir snippet.
4. Fusion: combine the two result lists with reciprocal rank fusion.
5. Rerank: a cross-encoder or LLM reranker on the top 50.
6. Output: top-k verses with scores and the evidence used.

Tools and models:

- Embeddings: BGE-M3 https://huggingface.co/BAAI/bge-m3 and multilingual-e5-large https://huggingface.co/intfloat/multilingual-e5-large
- Arabic models and tools: AraBERT https://huggingface.co/aubmindlab/bert-base-arabertv2 and CAMeL Tools https://github.com/CAMeL-Lab/camel_tools
- Framework: Sentence Transformers https://www.sbert.net
- Vector search: FAISS https://github.com/facebookresearch/faiss or Chroma https://www.trychroma.com
- BM25: rank_bm25 https://github.com/dorianbrown/rank_bm25

6. Module 2: Knowledge-weight ranking

Goal: for a topic t, give every candidate verse v a score KW(v, t) between 0 and 1 that says how much knowledge that verse carries on the topic, and rank by it.

Signals (each normalized to 0 to 1):

Signal	Meaning	How to compute
S1 Semantic similarity	Verse meaning matches the topic	Embedding cosine similarity
S2 Lexical match	Topic words or roots appear	BM25 score or root overlap
S3 Tafsir support	Classical tafsir discusses this topic for this verse	Similarity between tafsir passages and the topic, normalized by passage length
S4 Explicitness	The verse states the idea directly rather than mentioning it in passing	Small classifier trained on scholar labels
S5 Centrality	Verse is connected to many other relevant verses	Degree or PageRank on a verse-similarity graph

```
KW(v,t) = w1*S1 + w2*S2 + w3*S3 + w4*S4 + w5*S5
starting weights: 0.30, 0.15, 0.25, 0.15, 0.15 (sum to 1)
```

Then learn the weights instead of guessing them. Have a knowledgeable person grade verses for each pilot topic (0 = unrelated, 1 = related, 2 = supporting, 3 = core) and train a learning-to-rank model such as LambdaMART in LightGBM https://lightgbm.readthedocs.io. Compare the learned ranking with your hand-set weights.

Present results in tiers: Core, Supporting, Related. Always show the score breakdown so users can see why a verse ranked where it did. This explainability is also strong material for your viva.

7. Module 3: Summary generation

1. Take the top verses by knowledge weight for the topic.
2. Group them into sub-themes with clustering or BERTopic https://maartengr.github.io/BERTopic/.
3. Attach the matching tafsir snippets.

4. Ask the LLM to write only from this material, in a fixed layout: overview, sub-themes, key verses, scholars' explanations with attribution, practical takeaways.
5. Insert verse text with code from the verse keys the model returns.
6. Run a citation check: every sentence must point to at least one retrieved verse or tafsir passage. Remove or flag unsupported sentences.

Support English and Urdu, and let the user choose short, medium or detailed length.

8. Module 4: Khutba generation

Use a template so the sensitive parts are fixed and only the explanatory parts are generated.

Section	Source
Opening praise and testimony of faith	Fixed, human-vetted template
Introduction to the topic	Generated from the topic summary
Two or three main points, each with a verse	Generated, verses inserted by code
Hadith (optional)	Only from graded collections, shown with source
Practical actions for the community	Generated, short and concrete
Closing supplication	Fixed, human-vetted template

Controls to offer: language, audience (youth, families, general), tone, and target length. A typical sermon is roughly 1,200 to 1,600 words, but confirm with your reviewer. Mark every output as a draft for review by an imam or scholar before delivery.

9. Module 5: Assessments

Question type	Example use
Multiple choice	Which verse addresses the topic directly?
True or false	Statement check against the tafsir summary
Fill in the blank	Complete a key idea from the translation
Match	Match verses to sub-themes

Question type	Example use
Short answer	Explain a verse in your own words
Reflection	Personal action plan (not graded for correctness)

Pipeline:

1. Pick verses by tier and sub-theme.
2. Generate questions as structured JSON: question, options, answer, explanation, verse key, level (Bloom's taxonomy: remember, understand, apply).
3. Validate with code: the answer must appear in the retrieved evidence, the options must be distinct, and the verse key must exist.
4. Auto-grade objective questions. Grade short answers against a rubric with evidence, and flag uncertain ones for a teacher.
5. Store results to measure how hard each question really is, then adjust.

10. Module 6: Micro-credential courses

Course template:

- Title and 3 to 5 learning outcomes
- 4 to 6 lessons, 10 to 15 minutes each (verses, tafsir summary, reflection)
- A short quiz after each lesson
- A final assessment with a pass mark (for example 80 percent)
- A badge or certificate on completion

Useful standards and tools:

- Open Badges 3.0 for verifiable badges: https://www.imsglobal.org/spec/ob/v3p0/
- Spaced repetition for revision: https://github.com/open-spaced-repetition/fsrs4anki

Example course: Patience (Sabr) in the Quran

1. What sabr means (language and roots)
2. Patience in hardship
3. Patience in obedience and restraint
4. Prophetic examples mentioned in the Quran
5. Applying sabr in daily life
6. Final assessment and badge

Adaptive path: if a learner scores low on a lesson, generate a revision set from the same verses before moving on.

11. Evaluation plan (this is what makes it a strong FYP)

Build a gold set first: 20 to 30 topics, each with verses graded by at least two annotators, and report their agreement (Cohen's kappa).

Module	Metrics	How
Search	Precision@10, recall@k, nDCG@10, MRR	Compare keyword, semantic, hybrid, hybrid plus rerank
KW ranking	Spearman or Kendall correlation with scholar grades	Ablation: remove one signal at a time
Summary	Faithfulness, coverage, reviewer rating (1 to 5)	Citation checker plus human review; Ragas https://docs.ragas.io can help
Khutba	Reviewer rubric, structure compliance, length	Scholar or imam review of a sample
Assessments	Answer-key accuracy, distractor quality, difficulty	Audit 100 questions by hand; pilot with classmates
Courses	Completion rate, usability score	Small pilot with a short questionnaire

One target is non-negotiable: zero invented or altered verses in output. Test it automatically by checking every verse in every output against the database.

12. Using MCP and agents while building

- Connect the official quran-mcp server to Claude Desktop or another MCP client and use it to spot-check your retrieval results against a trusted source. Remote endpoint: https://mcp.quran.ai
- Use a coding agent (Claude Code or Cursor) to scaffold the pipeline, indexing scripts, and evaluation harness.
- Use Context7 for current library documentation.
- Optional advanced goal: expose Mudabbir itself as an MCP server so other AI apps can query it. Docs: https://modelcontextprotocol.io and the Python SDK https://github.com/modelcontextprotocol/python-sdk

Give agents narrow permissions (your project folder only) and review every change before you commit.

13. App screens (design plan)

1. Home: search box, topic suggestions, language switch.
2. Results: verses in tiers (Core, Supporting, Related), each with a score breakdown and a link to tafsir.
3. Topic summary: sub-themes, key verses, scholars' explanations, download as PDF.
4. Khutba builder: choose audience, length and language, edit the draft, export.
5. Quiz: question types, instant feedback with the verse reference.
6. Course: lessons, progress bar, badge.
7. Admin or reviewer view: approve labels and flag bad outputs.

Design tips: right-to-left support for Arabic and Urdu, a readable Arabic font such as Amiri or Noto Naskh Arabic, large text, and a visible disclaimer. Prototype in Figma or ask for a mockup before coding.

14. Suggested timeline (adjust to your semester)

Phase	Weeks	Output
1	1 to 3	Data loaded, keyword baseline, repo and evaluation harness
2	4 to 6	Hybrid search, reranking, gold set for pilot topics
3	7 to 8	Knowledge-weight ranking and ablations
4	9 to 11	Summary and khutba generation with guardrails
5	12 to 13	Assessments and micro-credential courses
6	14 to 16	Full evaluation, UI polish, report, viva rehearsal

15. Tips and tricks

1. Build the keyword baseline and the evaluation harness before anything fancy. You need a number to beat.
2. Start with 5 pilot topics, get them excellent, then scale.
3. Use verse keys (surah:ayah) as permanent IDs everywhere.
4. Normalize Arabic for searching (diacritics, alef and ya variants) but always display the original Uthmani text unchanged.

5. Embed verse plus translation plus a tafsir snippet. A short verse alone carries too little context. Also try windows of neighboring verses, since themes often span several.
6. Cache embeddings and version your datasets and models, for example with DVC https://dvc.org.
7. Keep prompts in version-controlled files and log every generation with the retrieved IDs it used.
8. Make the LLM return structured JSON and validate it with Pydantic https://docs.pydantic.dev before using it.
9. Show score breakdowns in the UI. Explainable ranking impresses examiners and builds user trust.
10. Run ablations (remove one signal, one model, one tafsir). Each ablation is a result for your report.
11. Split the team by module and agree on interfaces first (what each module takes in and returns).
12. Prepare the viva demo around three topics end to end, and include one honest failure case with your analysis of why it failed.
13. Keep API keys in a .env file that is never committed, and collect no personal user data.
14. Use free GPUs (Colab, Kaggle) for embedding and fine-tuning experiments.
15. Write short decision notes in the repo (what you chose and why). They become your methodology chapter.

16. Risks and how to handle them

Risk	Mitigation
Model invents or alters a verse	Verse text inserted by code from the database; automatic verification of every output
Wrong or one-sided interpretation	Tafsir from recognized sources with attribution; show multiple viewpoints; scholar review; no rulings
Hadith misuse	Graded collections only, or omit in version 1
License problems with tafsir or translations	Check terms for each source and cite them
Weak Arabic or Urdu performance	Evaluate per language; fall back to the best-performing model

Risk	Mitigation
Scope too large	Finish modules 1 to 3 well before polishing modules 4 to 6

17. Next steps this week

1. Agree on the definition of knowledge weight with your supervisor.
2. Choose 5 pilot topics and recruit a knowledgeable annotator.
3. Set up the repository and load the Arabic text and one translation.
4. Build the keyword search baseline and the evaluation script.
5. Connect quran-mcp and use it to cross-check a few results.